

STAT 432: Homework 06

Jacob Jacobson

October 4, 2026

Question 1: Compare KNN and logistic regression

I used the supplied split generated with seed 432: 398 observations for training and 171 for testing. I estimated each predictor's mean and sample standard deviation from the training set only, then applied those same transformations to both sets. Logistic regression was fit with an intercept and no penalty. For KNN, the estimated malignancy probability was the fraction of malignant outcomes among the k nearest training observations, using Euclidean distance and equal weights.

Model	Test AUC
Logistic regression	0.9746
KNN, $k = 5$	0.9329
KNN, $k = 15$	0.9480
KNN, $k = 25$	0.9599
KNN, $k = 35$	0.9727
KNN, $k = 45$	0.9741

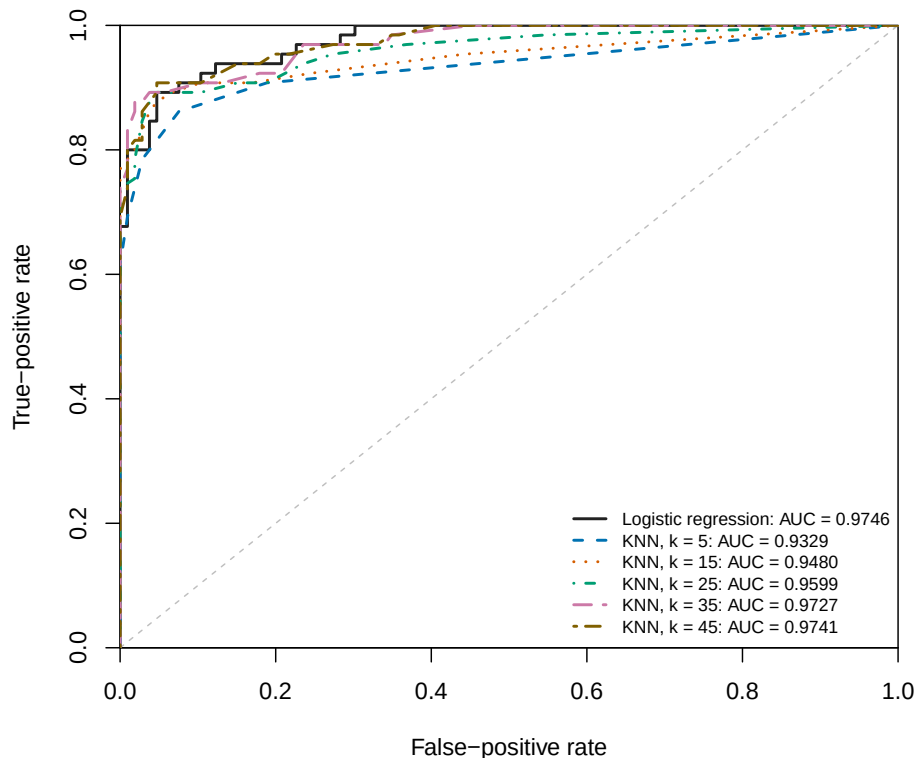


Figure 1: Test ROC curves for all six fits, evaluated on the same 171 test observations.

I prefer logistic regression for this split. It has the largest test AUC, 0.9746, while remaining a simple fitted model. KNN with $k = 45$ is nearly tied at 0.9741, so the result does not show a meaningful universal advantage for logistic regression. The larger neighborhoods rank the test cases better than the smaller

ones here. This conclusion is limited to one test split; AUC measures ranking over all cutoffs and does not select a classification cutoff or incorporate unequal costs of false negatives and false positives.

Question 2: Training and test ROC curves

I reused the split and training-based standardization from Question 1. For each training prediction, the target observation was included among its own neighbors as required.

k	Training AUC	Test AUC
1	1.0000	0.8764
3	0.9905	0.9138
5	0.9847	0.9329
10	0.9862	0.9446
20	0.9880	0.9586

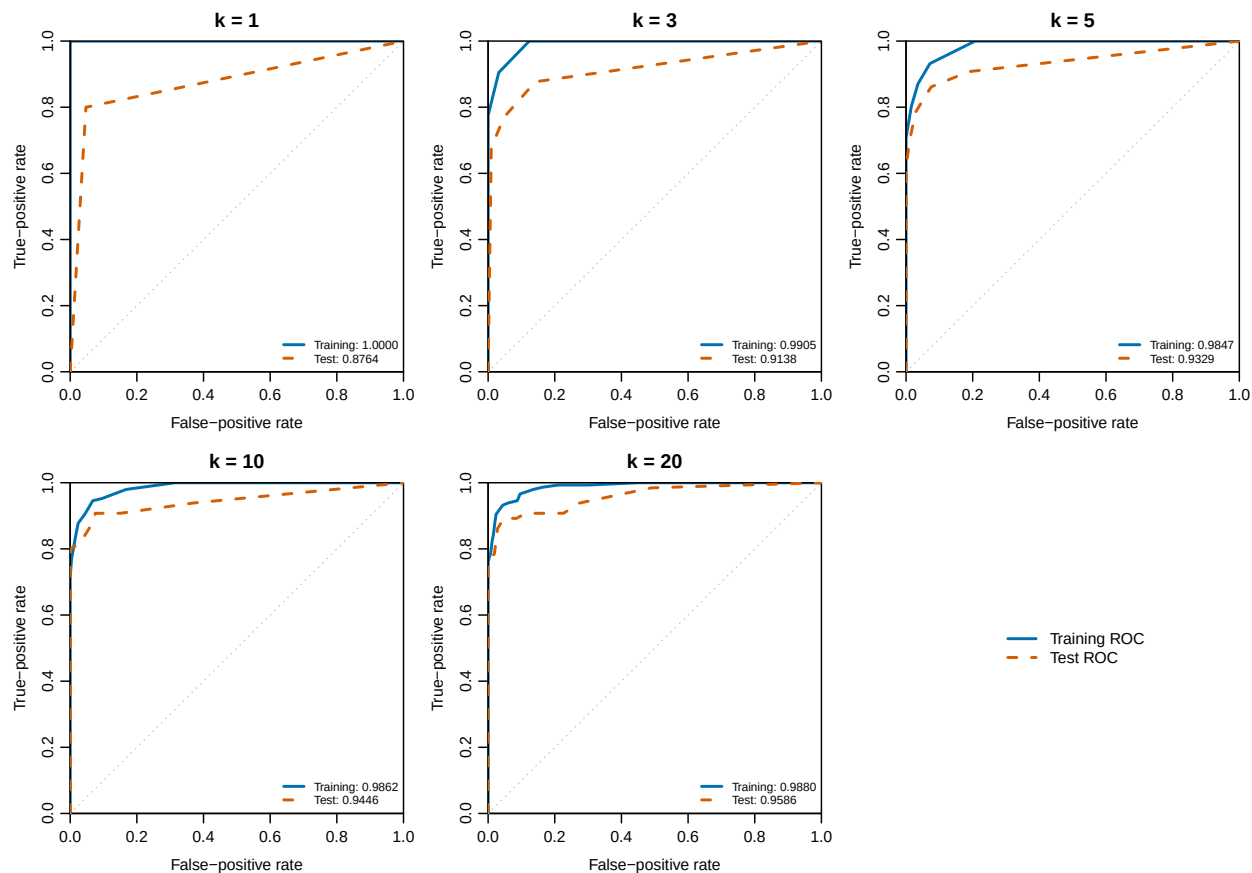


Figure 2: Training and test ROC curves by neighborhood size. Each training observation is included among its own neighbors.

At $k = 1$, every training observation is its own closest neighbor. Its fitted probability is therefore exactly its observed label, so the training AUC is 1.0000. A new test observation cannot use its own unknown label in this way, and its AUC is only 0.8764. The roughly 0.124 training-test gap is direct evidence that the apparent training performance is overly optimistic.

As k grows, each individual label has less influence and the probability estimates become smoother. Test AUC rises across the displayed values, reaching 0.9586 at $k = 20$, while the training-test gap narrows to about 0.029. Training AUC is not monotone: after falling through $k = 5$, it increases slightly for $k = 10$ and $k = 20$ because AUC depends on ranking, which smoothing can sometimes improve. In general

neither test AUC nor the gap must change monotonically; a neighborhood that becomes too large can average across genuinely different regions and add bias.

For 1NN, the test probabilities are only zero or one, so its ROC curve has a single nontrivial operating point. Its perfect training ROC reflects reuse of training outcomes, not an ability to generalize.

Question 3: Peer evaluation of Homework 5

Selected report and question. BrianChao623, Homework 5, Question 2 (Compare KNN with lasso). The Setting 1 and Setting 2 analyses repeat the same 5NN and cross-validated lasso fitting code nearly line for line, with only the variable names changed.

Assessment. i believe that your homework could be improved by removing redundant code! there is redundant and near-duplicated code in your knn analysis, and i think that cleaning up your code and reviewing code without AI analysis would be a good way to fix this!

Correction. The fitting steps (5NN, fold assignment, `cv.glmnet`, prediction at `lambda.min`, and counting nonzero slopes) go in one function that is called once per setting, and the response function is written once. The data are generated in the same order as in the original code, and the responses are created before each function call (R evaluates arguments lazily), so the random-number stream and the reported table are unchanged.

```
library(kknn)
library(glmnet)
set.seed(54782)
n <- 400
n_test <- 1000
p <- 30
lambda <- 10^seq(0, -4, length.out = 41)

f <- function(x) 0.5 * x[, 1] + sin(x[, 2]) - 0.3 * x[, 3]^2
gen_y <- function(x) f(x) + rnorm(nrow(x), sd = 0.1)

# Fit 5NN and CV lasso on one setting; return one row of the results table
fit_setting <- function(x, y, x_test, y_test) {
  knn_pred <- kknn(y ~ ., train = data.frame(x = x, y = y),
                  test = data.frame(x = x_test), k = 5,
                  kernel = "rectangular", scale = FALSE)$fitted.values
  fold <- sample(rep(1:10, each = n / 10))
  cv <- cv.glmnet(x, y, alpha = 1, lambda = lambda, foldid = fold,
                type.measure = "mse", standardize = TRUE)
  lasso_pred <- predict(cv, newx = x_test, s = "lambda.min")
  c(KNN_MSE = mean((y_test - knn_pred)^2),
    Lasso_MSE = mean((y_test - lasso_pred)^2),
    Lambda = cv$lambda.min,
    Nonzero = sum(coef(cv, s = "lambda.min")[-1] != 0))
}

# Setting 1: independent covariates
x_test <- matrix(rnorm(n_test * p), n_test, p)
x <- matrix(rnorm(n * p), n, p)
y <- gen_y(x)
y_test <- gen_y(x_test)
res_ind <- fit_setting(x, y, x_test, y_test)

# Setting 2: equicorrelated covariates (Sigma_jl = 0.8)
sigma <- matrix(0.8, p, p)
diag(sigma) <- 1
R <- chol(sigma)
x_cor <- x %*% R
x_test_cor <- x_test %*% R
y_cor <- gen_y(x_cor)
```

```
y_test_cor <- gen_y(x_test_cor)
res_cor <- fit_setting(x_cor, y_cor, x_test_cor, y_test_cor)

results <- data.frame(Setting = c("Independent", "Correlated"),
                      rbind(res_ind, res_cor), row.names = NULL)
knitr::kable(results, digits = 3)
```

Posted GitHub issue. <https://github.com/BrianChao623/stat432-fall2026/issues/2>